Agentic Software: AI 에이전트는 어떻게 소프트웨어 패러다임을 재구성하는가

원문: Zhenfeng Cao, *Agentic Software: How AI Agents Are Restructuring the Software Paradigm*, Lingxi Intelligent Investment (Shenzhen), 2026년 8월 24일 (arXiv:2606.05608)

한눈에 보는 핵심 주장

이 논문은 "AI 에이전트의 등장은 소프트웨어 공학의 도구 하나가 추가된 것이 아니라, '소프트웨어란 무엇인가'라는 정의 자체를 재구성하는 사건"이라고 주장합니다. 저자는 세 가지 핵심 명제를 제시합니다.

1. **원리적 필연성** — 에이전트 패러다임은 시장의 취향이 아니라 복잡도 스케일링 법칙에서 비롯된 필연적 귀결이다. 전통적 소프트웨어는 모든 의사결정 로직을 인간이 명시적으로 코드화해야 하지만, LLM 기반 에이전트는 추론을 모델에 위임함으로써 비선형적으로 복잡도를 다룰 수 있다.
2. **소프트웨어의 재정의, 대체가 아님** — "AI → 소프트웨어 → 결과"에서 "에이전트 → 결과"로의 전환은 소프트웨어를 없애는 것이 아니라, 에이전트 자체가 소프트웨어가 되면서 중간 매개체를 붕괴시키는 것이다. 이는 소프트웨어 전달 방식의 세 번째 거대한 패러다임 전환이다.
3. **새로운 학문 분야의 출현** — "에이전틱 엔지니어링(Agentic Engineering)"은 고유한 개념·도구·지표를 갖춘 독자적 실무 분야로 떠오르고 있으며, 그 실무자는 "더 나은 프로그래머"가 아니라 의도 설계자(intent architect), 에이전트 조정자, 결과 감사자라는 전혀 다른 역할을 맡는다.

---

1. 서론: 왜 지금 패러다임 전환인가

1968년 NATO 소프트웨어 공학 컨퍼런스는 임기응변식 프로그래밍으로는 감당할 수 없는 복잡도의 위기에서 출발했습니다. 이후 50년간 구조적 설계, 모듈화, 형상관리, 체계적 테스트 등 엄밀한 방법론으로 이 복잡도를 다스려 왔고, 워터폴에서 애자일로, 모놀리스에서 마이크로서비스로, 수동 배포에서 CI/CD로 발전해 왔습니다.

그러나 브룩스(Brooks)가 『맨먼스 미신』에서 지적했듯, 소프트웨어의 복잡도는 다른 공학 분야와 근본적으로 다른 방식으로 확장됩니다. 다리나 회로와 달리 소프트웨어에는 '제조' 단계가 없어 설계 자체가 곧 제품이며, 기능이 하나씩 늘어날 때마다 상태와 상호작용의 조합이 폭발적으로 늘어납니다. 브룩스는 이를 구현의 부산물인 "우발적 복잡도"와 문제 자체에 내재한 "본질적 복잡도"로 구분했는데, 후자는 지금까지 줄어든 적이 없습니다.

저자는 AI 에이전트가 이 전제 자체를 확장한다고 봅니다. 에이전트도 하드웨어에서 실행되고 데이터를 처리하고 결과를 낸다는 점에서 여전히 '소프트웨어'이지만, 결정적 차이는 의사결정 로직이 미리 쓰이는 것이 아니라 실행 시점에 동적으로 생성된다는 점입니다. LLM이 과제를 이해하고, 하위 과제로 분해하고, 그때그때 코드를 생성해 실행한 뒤 필요 없어지면 버릴 수 있게 되면서, 코드의 위상은 '시스템 그 자체'에서 '추론을 위한 일회성 도구'로 바뀝니다. 저자는 이를 아날로그 회로에서 저장 프로그램 방식 컴퓨터로의 전환에 비견합니다.

---

2. 원리적 분석: 전통적 소프트웨어 vs. 에이전트 시스템

2.1 전통적 소프트웨어의 정의

논문은 전통적 소프트웨어 시스템을 3원소 튜플 **S = (C, D, E)** 로 정식화합니다.

- **C**: 계산 자원 (CPU, 메모리, I/O)
- **D**: 소스코드에 인코딩된 결정론적 의사결정 규칙의 집합
- **E**: D를 입력에 대해 평가해 출력을 만드는 실행 환경

핵심은 **D가 실행 시점에 대해 정적(static)**이라는 것 — 모든 의사결정 로직이 어떤 입력을 만나기도 전에 인간 엔지니어에 의해 이미 다 쓰여 있어야 합니다. 그 결과 기능 추가, 버그 수정, 환경 변화 대응 하나하나가 (a) 필요한 변경 이해, (b) D 내 정확한 위치 파악, (c) 회귀 없는 수정, (d) 정합성 검증이라는 인간 개입을 요구하며, 변경 비용은 D의 크기와 내부 의존성 밀도에 비례해 커집니다.

2.2 복잡도 장벽

n개의 컴포넌트가 서로 상호작용할 수 있는 시스템에서, 가능한 상호작용 위상의 수는 2^(n choose 2)로, 즉 $\Theta(2^{(n^2)})$ 규모로 커집니다. 실제 시스템이 모든 조합을 실현하지는 않지만 복잡도의 상한은 초지수적으로 증가하는 반면, 이를 다루는 인간의 인지 능력은 사실상 상수입니다. 계층적 분해나 모듈 인터페이스, 캡슐화 같은 전통적 대응은 이 비례상수를 줄일 뿐, 점근적 증가 추세 자체를 바꾸지는 못합니다.

2.3 에이전트 시스템의 형식적 모델

에이전트 시스템은 4원소 튜플 **A = (M, $\mathcal{T}$, $\mathcal{M}$, Π)** 로 정의됩니다.

- **M**: 추론 엔진 역할을 하는 대규모 언어모델
- $\mathcal{T}$: 실행 가능한 도구 집합 (코드 인터프리터, API, 데이터베이스, 파일시스템)
- $\mathcal{M}$: 메모리 서브시스템 (단기 컨텍스트, 장기 벡터 스토어)
- **Π**: 사용자 의도를 행동 시퀀스로 분해하는 계획 메커니즘

시스템은 "$a_t \leftarrow M(s_t, \mathcal{M})$, $s_{t+1} \leftarrow \text{exec}(a_t)$"를 반복 실행하며 동작합니다. 핵심은 **의사결정 로직이 실행 시점에 생성된다는 점**입니다. LLM은 중간 결과에 따라 동적으로 코드를 만들고, 도구를 호출하고, 행동을 조정하는데 이 중 어느 것도 미리 프로그래밍되어 있지 않습니다. 여기서 생성되는 코드는 시스템 그 자체가 아니라, 필요에 따라 만들어졌다 버려지는 일시적 산출물입니다.

저자는 이를 카파시(Karpathy)의 "Software 2.0" 개념(신경망이 손으로 짠 로직을 학습된 가중치로 대체)의 연장선으로 보되, 한 걸음 더 나아간 것으로 설명합니다. Software 2.0에서는 신경망이 프로그램을 대체하지만, 에이전트 시스템에서는 신경망이 더 큰 추론 목표를 위해 필요할 때마다 즉석에서 프로그램을 "작성"합니다. 이는 추론과 도구 사용을 교차시키는 ReAct 프레임워크, 명시적 중간 추론 단계로 LLM의 잠재 능력을 끌어내는 Chain-of-Thought 프롬프팅 연구와도 맥을 같이합니다.

2.4 왜 스케일링 방식이 근본적으로 다른가

해결 공간의 크기가 N인 과제 T가 있다고 할 때, 전통적 패러다임에서는 인간 엔지니어가 이 공간을 직접 탐색해 해법을 찾고 정적 프로그램으로 인코딩해야 하며, 인간의 인지 용량 C_H는 사실상 고정되어 있으므로 N이 C_H를 넘어서면 그 과제는 어떤 현실적 비용으로도 처리할 수 없게 됩니다. 반면 에이전트 패러다임에서는 LLM의 유효 용량 C_M이 모델 크기와 학습 연산량에 따라 함께 커지고, 계획 메커니즘 Π가 과제를 하위 문제로 쪼개며, 코드는 실제로 필요한 해법 경로에 대해서만 생성됩니다. 즉 에이전트 패러다임은 **해결 가능한 문제의 용량을 인간의 인지 한계로부터 분리**시키며, 이는 10% 수준의 개선이 아니라 어떤 문제를 경제적으로 풀 수 있는가에 대한 질적 전환이라는 것이 저자의 주장입니다.

---

3. SaaS에서 AaaS로: 세 번째 패러다임 전환

3.1 소프트웨어 전달 방식의 세 세대

세대	핵심 메커니즘	복잡도를 떠안는 주체	수익 모델	대표 사례
Software 1.0 (로컬)	코드·데이터가 온프레미스에서 실행	최종 사용자 (설치·유지보수)	라이선스 판매	Microsoft, Oracle
Software 2.0 (SaaS)	코드·데이터가 클라우드에서 실행	벤더 (인프라·업데이트)	구독	Salesforce, AWS
Software 3.0 (AaaS)	에이전트가 클라우드에서 자율적으로 동작	에이전트 (이해·구축·실행)	결과 기반	OpenAI, Anthropic

각 전환은 동일한 패턴을 따릅니다 — 복잡도를 가장 잘 감당할 수 있는 쪽이 그것을 떠안고, 가장 감당하기 어려운 쪽은 거기서 해방됩니다. SaaS가 기업을 서버실 운영에서 해방시켰다면, AaaS는 "어떻게 결과를 만들 것인가"를 지정할 필요 자체에서 기업을 해방시키고, 오직 "무엇을 원하는가"만 specify하면 되는 상태로 나아간다는 것입니다.

3.2 "AI → 소프트웨어 → 결과" 파이프라인의 구조적 한계

지금까지 주류였던 "AI 보조 개발" — LLM으로 인간 엔지니어의 코드 작성을 더 빠르게 돕되 전통적 소프트웨어 생애주기는 그대로 유지하는 방식 — 은 세 가지 구조적 약점을 갖습니다.

1. **병목의 지속**: 설계 판단, 아키텍처, 통합 테스트, 배포의 크리티컬 패스에는 여전히 인간 엔지니어가 남아 있다. AI는 구현의 일부인 코드 생성을 가속할 뿐, 어느 단계에서도 인간을 완전히 대체하지 못한다.
2. **복잡도 상한선 불변**: 최종 산출물은 여전히 전통적 시스템 S=(C,D,E)이며, D의 크기에 비례해 복잡도가 커지고 수정에는 여전히 인간의 이해가 필요하다. AI는 D를 만드는 속도를 다소 높였을 뿐이다.
3. **반복 지연**: AI 보조가 있어도 기능 변경은 요구사항 → 설계 → 코드 → 테스트 → 배포라는 전체 사슬을 거쳐야 하며, 이 지연은 인간의 소통·조율 속도 아래로 줄어들 수 없다.

3.3 "에이전트 → 결과": 에이전트 자체가 소프트웨어가 되는 모델

대안적 패러다임은 소프트웨어와 그 실행 사이의 구분을 붕괴시킵니다 — **에이전트 자체가 소프트웨어**입니다. 인간이 만든 정적 시스템이 결과를 산출하는 대신, 에이전트가 추론하고 코드를 생성·실행하며 결과를 전달하는 통합 루프 안에서 동작합니다.

1. 인간이 에이전트에게 의도와 제약조건을 전달한다.
2. 에이전트가 자율적으로 계획을 세우고 (필요하면 코드를 생성해) 실행·검증하며 결과를 전달한다.
3. 인간이 결과를 감사하고 피드백을 제공한다.

이 모델에서 에이전트는 소프트웨어이자 운영자입니다. 수천 줄의 코드를 생성하고, DB 쿼리를 실행하고, 외부 API를 호출하고, 시각화를 만들어낼 수 있지만 이 모두는 일시적이며, 지속되는 것은 중간 산출 코드가 아니라 에이전트의 역량 그 자체입니다. LangChain의 Kumar와 Ramagopal은 이를 다음과 같이 표현했다고 인용됩니다: "AI 코딩 에이전트는 단일 세션 안에서 의도를 코드로 옮기는 데 탁월하다. 에이전틱 엔지니어링은 그보다 더 높은 추상화 층위에서 작동하는 제어 평면(control plane)으로, 팀 간 워크플로를 조율하고 에이전트 간 장기 메모리를 유지하며 전체 소프트웨어 전달 생애주기에 걸친 상태와 추적성을 관리한다."

---

4. 에이전틱 엔지니어링: 학문 분야의 확장

4.1 분야 정의

2026년 4월 LangChain이 공식 제시한 "에이전틱 엔지니어링"은 "AI 에이전트가 정의된 역할, 공유 메모리, 통합 관측 계층을 갖춘 디지털 팀원으로서 기능하며 소프트웨어를 전체 전달 파이프라인 전 과정에 걸쳐 이끄는 다중 에이전트 조정 모델"로 정의됩니다. 저자는 이것이 소프트웨어 공학을 대체하는 것이 아니라 **확장**하는 것이라고 봅니다 — 에이전트 자체가 소프트웨어이므로, 에이전트 시스템을 만들고 배포하고 거버넌스하는 일이 곧 이 학문의 다음 프런티어라는 것입니다.

Wang 등(2024)은 소프트웨어 공학용 LLM 기반 에이전트의 기초 분류체계를 제시하며, 지각(Perception, 멀티모달 입력 처리) · 메모리(의미·일화·절차 기억) · 행동(내적 추론 + 외부 도구 사용)이라는 세 핵심 모듈이 LLM 추론 코어에 의해 조율되는 구조를 제안했습니다.

이 구조의 구체적 사례로 논문은 Nous Research의 오픈소스 프레임워크 **Hermes Agent**를 듭니다. 가장 특징적인 것은 닫힌 학습 루프입니다 — 복잡한 과제를 마친 뒤 에이전트가 재사용 가능한 "스킬"(파라미터화된 절차 모듈)을 스스로 만들고, 이후 사용 과정에서 부족함이 발견되면 스스로 패치하며 개선해 나간다는 것입니다. FTS5 기반 대화 검색과 LLM 요약을 통한 세션 간 일화 기억, 서브에이전트 위임 메커니즘도 초기 단계의 다중 에이전트 조정을 보여주는 사례로 언급됩니다.

4.2 전통적 소프트웨어 공학 vs. 에이전틱 엔지니어링

차원	전통적 소프트웨어 공학	에이전틱 엔지니어링
핵심 산출물	소스코드 (정적)	에이전트 시스템 (동적)
통제 중심	인간 엔지니어	LLM 추론 엔진
의사결정 메커니즘	사전 설계된 로직	실행 시점에 생성되는 추론
개발 주기	선형 (설계→코드→테스트)	자율적 반복 루프
인간의 역할	코드 작성자	의도 설계자·조정자·감사자
복잡도 상한	인간 인지 (O(1))	모델 용량 (연산량에 따라 성장)
산출 단위	작동하는 소프트웨어	전달된 결과(outcome)
오류 처리	프로그래머가 정의	모델이 적응적으로 처리
진화 방식	수동 리팩터링	자기 수정(self-modification)

4.3 재정의되는 인간의 역할

가장 중대한 변화는 인간의 역할입니다. 전통적 패러다임에서 인간의 가치는 정확하고 효율적인 코드를 만드는 능력으로 측정되었지만, 에이전틱 패러다임에서는 코드 생성 능력 자체가 상품화(commoditize)됩니다. 새로운 인간 차별화 요소로 저자는 다음을 꼽습니다.

- **의도 명확화(intent articulation)**: 에이전트가 의도치 않은 결과를 내지 않고 자율적으로 동작할 수 있을 만큼 충분히 명확하게 목표와 제약을 specify하는 능력
- **아키텍처적 감독**: 여러 에이전트가 어떻게 조율되어야 하는지, 어떤 메모리를 공유해야 하는지, 어디에서 인간의 판단이 개입해야 하는지를 시스템 수준에서 이해하는 능력
- **품질 보정(quality calibration)**: "좋다"는 것이 무엇인지 정의하고, 에이전트가 스스로 교정하는 데 쓸 수 있는 평가 체계를 구축하는 능력
- **윤리적 거버넌스**: 에이전트의 행동이 조직의 가치, 법적 요구, 사회적 기대에 부합하도록 보장하는 능력

저자는 에이전트 조율(orchestration)을 능숙히 다루는 실무자의 생산성 배수가 전통적인 "10배 엔지니어" 벤치마크를 훨씬 뛰어넘을 것이며, 이는 더 빠른 타이핑이 아니라 복잡한 결과를 향해 에이전트 무리를 조율하는 능력에서 온다고 전망합니다.

---

5. 실증적 근거와 현재의 한계

5.1 성과를 뒷받침하는 사례들

- **SWE-bench Verified**: Ma 등(2024)의 오픈 모델 Lingma SWE-GPT 72B는 SWE-bench Verified에서 GitHub 이슈의 30.20%를 해결해 GPT-4o의 31.80%에 근접했으며, 7B 소형 모델도 18.20%를 기록해, 정적 코드가

아니라 개발 프로세스 데이터로 학습하면 소형 모델도 유의미한 자동화된 소프트웨어 엔지니어링을 수행할 수 있음을 보였습니다. 이는 약 6배 큰 Llama 3.1 405B 대비 22.76%의 상대적 개선입니다.

- **다중 에이전트 조정**: LangChain의 Kumar와 Ramagopal은 20개 이상의 기업 디버깅 워크플로에 조율된 에이전트 무리를 투입한 파일럿 연구에서, 근본 원인 규명 시간을 93% 단축하고 한 달 만에 200시간 이상의 엔지니어링 공수를 절감했다고 보고합니다. 이 개선은 개별 에이전트의 성능이 아니라 에이전트 간 컨텍스트 공유, 병렬 조사, 상호 검증 같은 **조율(orchestration)** 자체에서 비롯되었다는 점이 강조됩니다.
- **자기 진화**: GitHub 스타 17만 9천 개 이상을 보유한 Hermes Agent는 "스킬 생성 → 사용 → 결함 감지 → 자가 패치"라는 인간 개입 없는 닫힌 루프를 프로덕션 환경에서 구현한 대표 사례로 제시됩니다.
- **일반화 가능성**: Wang 등(2024)이 정리한 수백 건의 연구는 요구사항 분석부터 아키텍처 설계, 코드 생성, 테스트, 디버깅, 배포, 유지보수까지 소프트웨어 생애주기 전반에 에이전트 패턴이 적용되고 있음을 보여줍니다.

5.2 여전히 남아 있는 한계

가장 냉정한 데이터는 **EvoClaw** 벤치마크(Deng 등, 2026)에서 나옵니다. 이 벤치마크는 개별 이슈 해결이 아니라, 커밋 히스토리 전반에 걸쳐 시스템 무결성을 유지하며 지속적으로 오류가 누적될 수 있는 **연속적 소프트웨어 진화** 과제를 에이전트에게 요구합니다. 논문이 인용하는 핵심 발견은 다음과 같습니다.

> "전체 성능 점수는 고립된 과제에서의 80% 이상에서 연속적 환경에서는 최대 38%까지 크게 떨어지며, 이는 장기 유지보수와 오류 전파에 대한 에이전트의 근본적인 어려움을 드러낸다." (Deng 등, 2026)

이 결과는 4가지 핵심 과제를 시사합니다.

1. **컨텍스트 드리프트**: 코드베이스가 효과적인 컨텍스트 윈도우를 넘어 커지면, 에이전트는 시스템 전반의 불변식과 의존관계에 대한 일관된 이해를 잃는다.
2. **오류 전파**: 초기 커밋의 작은 오류가 이후 작업에서 누적적으로 실패를 일으키는데, 에이전트는 이런 오류 연쇄를 감지하고 복구하는 견고한 메커니즘을 아직 갖추지 못했다.
3. **기술 부채 인식 부족**: 에이전트는 설계 결정의 장기적 비용을 모델링하지 못하고, 유지보수성을 고려하지 않은 채 당장의 과제 완료만을 최적화한다.
4. **검증 신뢰도**: 자동화된 테스트는 여전히 불완전하며, 에이전트는 테스트를 통과시키면서도 새로운 입력에서만 드러나는 미묘한 의미적 오류를 만들어낼 수 있다.

(12개 최신 모델·4개 에이전트 프레임워크를 대상으로 한 평가에서, 고립 과제 성공률 82% → 연속 진화 과제 38%로 **54%p 하락**한 그래프가 함께 제시됩니다.)

저자는 이 격차가 근본적인 한계가 아니라 컨텍스트 관리, 메모리 아키텍처, 검증 메커니즘의 현재 한계를 반영하는 것으로 보며, 지금은 에이전틱 엔지니어링이 "보강(augmentation)" 패러다임으로서는 이미 현실적이고 변혁적이지만, 완전 자율 소프트웨어 개발이 프로덕션에서 신뢰할 수 있게 되기까지는 앞으로 수년간의 집중적 연구가 더 필요한 시점이라고 평가합니다.

---

6. 진화 로드맵 (4단계)

단계	에이전트 역량	핵심 기술	인간의 역할	대표 시스템
I. 도구 보강형 (2023-2025)	코드 완성, 단일 이슈 수정, 간단한 스크립트 생성	인컨텍스트 러닝, RAG	작성자 + 검토자	GitHub Copilot, Claude Code
II. 단일 과제 자율형 (2025-2027)	종단 간 기능 구축, 디버깅, 기본적 시스템 유지보수	계획+도구 사용, 자기수정	의도 설계자 + 감사자	Devin, OpenHands
III. 다중 에이전트 팀 (2026-2029)	대규모 시스템을 위한 조율된 무리, 전체 생애주기 관리	공유 메모리, 역할 전문화, 오케스트레이션	PM + 아키텍트 + 감사자	LangChain 오케스트레이션, MetaGPT
IV. 자기 진화 생태계 (2028+)	자율적 발견·학습·재생산·적응	메타러닝, 자기수정, 생태계 거버넌스	목표 설정자 + 윤리 거버너	(전망 단계) AGI 어시스턴트

- **1단계(현재 주류)**: 에이전트는 인간이 주도하는 워크플로 안의 보조자다. 코딩 분야에서 특히 두드러진 성과를 냈지만, 문제 분해·아키텍처 설계·정합성 검증은 여전히 인간의 몫이다.
- **2단계**: 명세부터 배포까지 완결된 과제를 에이전트가 스스로 소유하기 시작한다. Devin, OpenHands 같은 시스템은 코드베이스를 자율 탐색하고 기능을 구현해 PR을 제출할 수 있음을 보여준다. 인간의 역할은 "직접 하기"에서 "무엇을 할지 specify하고 결과를 검증하기"로 이동한다.
- **3단계**: 전문화된 에이전트들이 인간 조직을 모방하듯 팀으로 조율한다 — PM 에이전트가 비즈니스 요구를 기술 명세로 옮기고, 아키텍트 에이전트가 구조를 설계하고, 개발자 에이전트가 컴포넌트를 구현하고, QA 에이전트가 테스트·검증한다. 공유 메모리와 관측가능성이 핵심 인프라가 된다.
- **4단계**: 에이전트가 자기 아키텍처를 스스로 개선하고, 새로운 문제 영역에 맞춰 전문화된 서브에이전트를 만들어내며, 인간 개입 없이 환경 변화에 적응한다. 이 단계에서는 "소프트웨어"와 "에이전트"의 구분이 완전히 사라지고, 인간의 관여는 윤리적 경계 설정, 가치함수 정의, 정합성 보장이라는 메타 수준 거버넌스로 옮겨간다.

---

7. 시사점과 제언

실무자에게

1. 코드 생산에서 **의도 엔지니어링(intent engineering)**으로 무게중심을 옮길 것 — 이제 가장 가치 있는 능력은 코드를 효율적으로 짜는 것이 아니라, 에이전트가 정확히 실행할 수 있을 만큼 명확한 맥락과 제약으로 과제를 표현하는 것이다.
2. **에이전트 오케스트레이션 역량**을 기를 것 — 여러 에이전트에 걸쳐 작업을 분해하고, 공유 메모리를 관리하고, 평가 기준을 설계하는 능력이 실무자를 차별화한다.
3. **관측가능성(observability) 인프라**에 투자할 것 — 에이전트의 추론 사슬을 추적하고, 환각을 탐지하고, 결과 품질을 측정하려면 전통적 소프트웨어와는 다른 새로운 도구가 필요하다.
4. "**인간은 루프 안에, 에이전트는 운전석에**" 라는 태도를 취할 것 — 현재 가장 효과적인 모델은 완전 자율도, 완전 인간 주도도 아니다. 에이전트가 실행을 맡고, 인간은 의도·비판적 판단·윤리적 감독을 맡아야 한다.

연구자에게

1. 장기 컨텍스트 상태 관리 (EvoClaw가 보여주듯, 긴 개발 시퀀스에서 에이전트는 일관성을 잃는다)
2. 개방형 환경에서의 검증 — 현재 벤치마크는 고립된 정확성만 테스트하지만, 실제 시스템은 시간에 걸친 안전성·신뢰성·유지보수성 보장이 필요하다
3. 대규모에서의 에이전트 정합성(alignment) — 에이전트가 팀으로 구성될수록 집단 행동이 인간 가치와 부합하도록 보장하는 일이 더 중요해지고 더 어려워진다
4. 경제 모델 — 구독·사용량 기반 과금 대신 결과 기반(해결된 이슈당, 배포된 기능당) 과금이 부상할 수 있으나, 인센티브 구조와 위험 배분에 대한 정교한 분석이 필요하다

조직에게

1. 에이전트 자동화에 적합한 워크플로(명확한 성공 기준, 잘 정의된 범위, 기존 테스트 인프라가 있는 과제)를 식별할 것
2. 정확성을 넘어 견고성·유지보수성·비즈니스 의도 부합성까지 측정하는 평가 프레임워크에 투자할 것
3. 에이전트 레버리지로 개인 생산성이 배가되는 만큼, "에이전트 오케스트레이터"로 구성된 소규모 팀이 대규모 개발자 팀을 대체할 수 있도록 채용·승진·경력 개발 구조를 재설계할 것

---

8. 결론

이 논문은 AI 에이전트의 등장이 "소프트웨어를 어떻게 만드는가"가 아니라 "소프트웨어란 무엇인가"에 대한 패러다임 전환이라고 결론짓습니다. "AI → 소프트웨어 → 결과"에서 "에이전트 → 결과"로의 전환은 소프트웨어를 없애는 것이 아니라, 의사결정 로직이 미리 인코딩되는 대신 실행 시점에 생성되고 인간 개입 없이 시스템 스스로 진화할 수 있는, 근본적으로 다른 종류의 소프트웨어로 대체하는 것입니다. 저자는 이를 소프트웨어 공학이라는 학문의 자연스러운 확장으로 보며, 그 종말이 아니라고 강조합니다.

이 전환은 원리적으로 뒷받침됩니다 — 전통적 소프트웨어는 모든 의사결정 로직을 인간이 명시적으로 인코딩해야 하며 그 작업의 복잡도는 시스템 규모에 따라 지수적으로 커지는 반면 인간의 역량은 고정되어 있습니다. 에이전틱 소프트웨어는 의사결정을, 학습 연산량에 따라 용량이 커지는 LLM에 위임함으로써 해결 역량을 인간의 인지 한계로부터 분리시킵니다.

그러나 저자는 아직 초기 단계임을 분명히 합니다. EvoClaw 같은 벤치마크는 고립된 과제 성능과 지속적인 자율 개발 성능 사이의 뚜렷한 격차를 드러내며, 지금 필요한 것은 야심 차면서도 신중하게 보정된 투자 — 에이전틱 소프트웨어를 새로운 패러다임으로 받아들이되, 완전 자율 시스템은 여전히 다년간의 연구 과제로 남아 있음을 인정하는 태도라고 말합니다.

에이전틱 엔지니어링은 소프트웨어 공학이라는 학문의 근본적 확장을 대표합니다. 그 실무자는 새로운 도구를 배운 프로그래머가 아니라, AI 에이전트를 복잡한 결과를 향해 이끄는 의도 설계자라는 새로운 종류의 전문가입니다. "낡은 소프트웨어 공학이 끝나는 것이 아니라, 더 큰 무언가로 성장하고 있다"는 것이 저자의 마지막 문장입니다.

참고문헌 (원문 인용 목록)

1. P. Naur & B. Randell (eds.), *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee*, NATO, 1968.
2. F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*, Addison-Wesley, 1975/1995.
3. A. Karpathy, "Software 2.0," Medium, Nov. 2017.
4. Y. Wang et al., "Agents in Software Engineering: Survey, Landscape, and Vision," arXiv:2409.09030, 2024.
5. Y. Ma et al., "Lingma SWE-GPT: An Open Development-Process-Centric Language Model for Automated Software Improvement," arXiv:2411.00622, 2024.
6. G. Deng et al., "EvoClaw: Evaluating AI Agents on Continuous Software Evolution," arXiv:2603.13428, 2026.
7. R. Kumar & P. Ramagopal, "Agentic Engineering: How Swarms of AI Agents Are Redefining Software Engineering," LangChain Blog, Apr. 2026.
8. J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS, 2022.
9. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023.
10. X. Wang et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR, 2024.
11. S. Hong et al., "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," ICLR, 2024.
12. T. B. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.
13. T. Guo et al., "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," IJCAI, 2024.
14. Nous Research, "Hermes Agent: The Self-Improving AI Agent," 2025-2026.

본 요약은 arXiv:2606.05608 (HTML 버전) 페이지 내용을 바탕으로 작성되었습니다.